DAMG 7245 - Big Data and Intelligent Analytics

Northeastern University | Spring 2026

MedSignal : Drug Safety Signal Detection Platform

Member	NUID
Samiksha Rajesh Gupta	002310743
Siddharth Rakesh Shukla	002303230
Prachi Ganpatrao Pradhan	002339613

Attestation: We attest that we haven't used any other students' work in our assignment and abide by the policies listed in the student handbook

Samiksha Rajesh Gupta: 33.3% | Prachi Ganpatrao Pradhan: 33.3% | Siddharth Rakesh Shukla: 33.3%

Table of Contents -

1. Title
2. Introduction
    2.1 Background
    2.2 Objectives
        Big Data Engineering
        Significant LLM Use
        Cloud-Native Architecture
        User-Facing Application
3. Project Overview
    3.1 Scope
        In Scope
        Out of Scope
    3.2 Stakeholders and End Users
4. Problem Statement
    4.1 Current Challenges
    4.2 Opportunities
5. Methodology
    5.1 Data Sources
    5.2 Technology Stack
5.3 Architecture
    System Overview
    Layer 1: Data Preparation
    Layer 2: Kafka Ingestion
    Layer 3: Spark Processing (Batch Mode)
    Layer 4: LangGraph Agent Pipeline
    Layer 5: Streamlit Interface
5.4 Data Processing and Transformation
    Batch vs stream processing
    Data formats
    Storage schemas
    Parallel processing strategy
    Data transformations in Branch 1
    Data transformations in Branch 2
    Embedding generation
5.5 LLM Integration Strategy
    Overview
    Prompt Design

- Retrieval-Augmented Generation
- Agentic Workflow
- API Usage Pattern
- How the LLM Contributes to the Problem

5.6 Guardrails and Human-in-the-Loop (HITL)
- Overview
- Input Guardrails
- Output Validation
- Human-in-the-Loop Design
- What is Not Automated

5.7 Evaluations and Testing
- Overview
- Pipeline Validation
- Golden Set Evaluation
- SafetyBrief Quality Evaluation
- Unit Tests

5.8 Proof of Concept
- POC 1 — Exploratory Data Analysis on FAERS 2023
- POC 2 — PubMed Abstract Ingestion into ChromaDB

6. Project Plan and Timeline
- 6.1 Milestones
- 6.2 Timeline

7. Team Roles and Responsibilities
- Samiksha — Data Engineering Lead
- Prachi — Data Processing and LLM Lead
- Siddharth — Infrastructure and Quality Lead
- Shared Responsibilities

8. Risks and Mitigation
- 8.1 Potential Risks
- 8.2 Mitigation Strategies

9. Expected Outcomes and Metrics
- 9.1 Key Performance Indicators
- 9.2 Expected Benefits

10. Token and Cost Report
- Overview
- GPT-4o Usage Points
- Total Token Estimate
- Cost Estimate

Cost Reduction Strategy
Monitoring
11. Conclusion
12. References

Title

MedSignal: A Scalable Pharmacovigilance Pipeline for Early Drug Safety Signal Detection Using FDA FAERS Data

Introduction

2.1 Background

When a drug gets approved, the safety testing does not stop. Clinical trials can only enroll so many patients, and rare side effects that show up in 1 in 10,000 people simply will not appear at that scale. The real picture of a drug's safety profile only becomes visible once it is being prescribed to millions of people in the real world.

This gap has had serious consequences. Vioxx was withdrawn in 2004 after it became clear it was significantly raising the risk of heart attacks. Simvastatin's link to muscle breakdown was known for years before the FDA restricted its highest dose in June 2011. More recently, 2023 saw a wave of FDA safety communications covering widely prescribed drugs including gabapentin, pregabalin, GLP-1 receptor agonists like Ozempic and Mounjaro, SGLT2 inhibitors like Jardiance and Farxiga, and dupilumab. In several of these cases, adverse event signals had been accumulating in reported data for months before the FDA formally communicated them.

The main system for capturing these signals in the US is the FDA Adverse Event Reporting System (FAERS). Healthcare professionals, patients, and manufacturers submit reports whenever they suspect a drug caused a harmful reaction. The database receives over two million reports a year and goes back decades.

Working with FAERS is not straightforward. Reports arrive quarterly as seven related files that need to be joined together. The same case gets resubmitted as updates across quarters, patients are often taking multiple drugs making it hard to identify the actual culprit, drug names appear in dozens of forms for the same active ingredient, and fields like event dates and outcomes are frequently missing. Reliable signal detection requires careful data engineering before any analysis can begin. As part of our preliminary work, we examined the actual 2023 FAERS ASCII files directly, confirming the schema, field completeness, and drug role distributions before designing the pipeline.

The standard method for detecting signals in pharmacovigilance is the Proportional Reporting Ratio (PRR). It measures whether a drug-reaction combination is being reported at a higher rate than you would expect by chance. A PRR above 1.0 with enough supporting cases is a signal worth investigating. But a number alone is not enough for an analyst to act on. It needs to be

supported by published evidence, outcome severity context, and a clear summary before it becomes clinically useful.

MedSignal addresses this end to end. It ingests raw FAERS data through a Kafka pipeline, processes it through Spark for normalization and PRR computation, and passes flagged signals through a three-agent LLM pipeline that retrieves supporting literature from PubMed and generates a structured safety brief. The system is evaluated against ten candidate drug-reaction pairs from 2023 FAERS data, selected through a preliminary proof-of-concept analysis based on documented FDA communications in 2023. Final PRR values and detection results will be produced by the full validated pipeline during the project.

2.2 Objectives

The goal of MedSignal is to build a working drug safety signal detection system that connects real FDA adverse event data to a reasoning pipeline capable of generating clinical safety assessments. We have four concrete things we want to deliver.

Big Data Engineering

The data side of this project is where most of our work lives. We are working with FDA FAERS quarterly data from 2023, which comes as four separate ASCII files per quarter: DEMO, DRUG, REAC, and OUTC. These files are not ready to use out of the box. They need to be joined on a shared identifier called primaryid, deduplicated because the same case can appear across multiple quarters as updates, and filtered so that only drugs genuinely suspected of causing a reaction are included in the analysis.

After this preparation, a thin Kafka producer script reads the four raw FAERS ASCII files and publishes records to four separate Kafka topics, one per file. No processing happens at this stage. Spark reads all four topics as static batch dataframes and processes them through two branches. The first branch performs the four-file join on primaryid, caseversion-based deduplication, Primary Suspect filtering, and RxNorm normalization, writing the result to the drug_reaction_pairs table in PostgreSQL. The second branch computes a Proportional Reporting Ratio across all drug-reaction pairs and flags combinations that cross a set of statistical thresholds. Based on our preliminary POC analysis, current thresholds are A >= 50, C >= 200, drug_total >= 1,000, and PRR >= 2.0. These values will be validated and may be adjusted during full pipeline testing. The final signals are written to PostgreSQL.

Before building anything else, we validate the pipeline against a known benchmark: the finasteride-depression PRR should come out to approximately 3.14 based on published pharmacovigilance research. If we cannot reproduce that number, something is wrong and we fix it before proceeding. More than 30% of project effort is allocated to data understanding, schema validation, pipeline correctness, and evaluation rigor.

Significant LLM Use

We have three agents working in sequence. Agent 1 reads the flagged signals from PostgreSQL and computes a StatScore based on PRR magnitude, case volume, and outcome severity from

FAERS death, hospitalization, and life-threatening flags. Agent 2 searches a ChromaDB vector store containing 1,500 to 2,000 PubMed abstracts fetched using targeted queries for each of our ten drug-reaction pairs, and computes a LitScore based on how closely the retrieved literature matches the signal. Agent 3 takes both scores and assigns a priority tier using a rule-based matrix that treats statistical evidence as the primary axis and literature support as the secondary differentiator. StatScore and LitScore are deliberately kept independent assigning weights to statistical versus literature evidence requires pharmacovigilance domain expertise the team does not have, and collapsing two meaningful dimensions into a single number would reduce transparency for the human reviewer. Agent 3 then calls an LLM to write a SafetyBrief summarizing the signal, citing PMIDs, and indicating a priority tier for reviewer consideration.

All flagged signals are routed to the HITL review queue regardless of their priority tier. Given the safety-critical nature of pharmacovigilance decisions, StatScore, LitScore, and the priority tier are presented to the reviewer as independent decision support inputs rather than as a single automated classification. This design acknowledges that automated scoring in a safety-critical domain should augment human judgment rather than replace it. The reviewer can approve, reject, or escalate each signal, with every decision recorded immutably in PostgreSQL.

Cloud-Native Architecture

We built this system so the components are loosely coupled and each one can be replaced or scaled without breaking the others. Kafka sits at the entry point and decouples ingestion from processing, which means if we ever connected a live hospital EHR feed instead of replaying FAERS files, the Spark and agent components would not need to change. Spark handles the heavy computation in parallel branches and scales horizontally by adding more workers. ChromaDB runs as a local persistent vector store. PostgreSQL holds all relational data with indexes on the most queried columns, keeping signal lookups fast even at 6 to 8 million rows. Everything runs in Docker so the environment is consistent across all three team members. Prometheus metrics capture pipeline observability including messages processed, Spark job duration, and HITL queue depth.

User-Facing Application

We are building the analyst interface in Streamlit across four pages. The signal feed shows all flagged drug-reaction pairs sorted by severity tier. The signal detail page shows the full SafetyBrief, cited PMIDs, and the SSS score breakdown. The HITL review queue is where CRITICAL and HIGH signals go for human review, with approve, reject, or escalate options, each decision recorded immutably in PostgreSQL. The evaluation page presents our main result: a detection lead time chart for our ten golden signal drugs alongside a precision-recall table showing how many signals the system correctly identified above the PRR threshold.

Project Overview

3.1 Scope

In Scope

Data Sources We are working with two data sources. The primary source is FDA FAERS 2023 quarterly data covering all four quarters, accessed as ASCII files from the FDA's public download portal. We use four of the seven available files: DEMO, DRUG, REAC, and OUTC. The secondary source is PubMed, from which we fetch abstracts using the NCBI Entrez API with targeted queries for each of our ten golden drug-reaction pairs.

ETL Pipeline The pipeline covers the full journey from raw FAERS ASCII files to flagged signals in PostgreSQL. This includes the four-file join on primaryid, caseversion-based deduplication, Primary Suspect role filtering, RxNorm-based drug name normalization via a pre-built cache, and PRR computation across all drug-reaction pairs. PubMed abstracts are embedded using a local sentence transformer model and loaded into ChromaDB as a one-time setup step.

LLM Components Three LangGraph agents handle signal assessment. Agent 1 computes a statistical score from FAERS-derived features. Agent 2 retrieves relevant PubMed literature from ChromaDB and scores it for relevance. Agent 3 generates a structured SafetyBrief using an LLM call, citing PMIDs and indicating a severity tier. All outputs are evaluated against a defined rubric.

Guardrails and HITL All flagged signals are routed to a human reviewer through the Streamlit HITL queue. The reviewer sees the SSS score, severity tier, SafetyBrief, and supporting literature before making a decision. Every decision is logged immutably in PostgreSQL. Output validation is handled using Pydantic schema enforcement on SafetyBrief structure before it is written to the database.

Evaluation We evaluate the system against ten golden drug-reaction pairs selected from 2023 FAERS data based on documented FDA safety communications. The primary evaluation metric is detection lead time: how many days before the FDA communication date our system first flagged each signal above the PRR threshold. Secondary metrics include PRR accuracy validated against a known benchmark, SafetyBrief quality scored against a four-criteria rubric, and HITL decision turnaround.

Out of Scope

The following are explicitly not part of this project:

- Real-time ingestion from live EHR systems or manufacturer feeds. The pipeline replays FAERS batch data through Kafka to simulate streaming behavior.
- Cloud deployment. The system runs locally using Docker containers. Cloud deployment is documented as a future extension.

- Coverage beyond ten golden signal drugs. PRR is computed across all drug-reaction pairs in the dataset but the evaluation and agent pipeline focus on the ten golden drugs.
- Reddit or social media data. PatientScore was considered and dropped. The SSS formula relies on FAERS statistical evidence and PubMed literature only.
- ClinicalTrials.gov integration. TrialScore was evaluated and removed as it provided near-constant values across all drugs and did not contribute meaningful signal differentiation.
- Automated regulatory reporting. MedSignal produces safety briefs and routes signals to human reviewers. It does not submit reports to any regulatory body.

3.2 Stakeholders and End Users

Primary User: Pharmacovigilance Analyst The main user of MedSignal is a PV analyst responsible for monitoring post-market drug safety. They interact with the Streamlit interface to review flagged signals, read SafetyBriefs, assess literature evidence, and make approve, reject, or escalate decisions. MedSignal reduces the time they spend manually querying FAERS and searching literature by surfacing pre-computed signals with supporting evidence already attached.

Secondary User: Drug Safety Team Lead A team lead or medical officer who oversees the HITL queue and reviews escalated signals. They benefit from the immutable decision log in PostgreSQL, which provides a full audit trail of every signal reviewed and every decision made.

Research Analyst: Monitors detection lag performance, token costs, false positive rates across the golden evaluation set

Problem Statement

4.1 Current Challenges

Data Fragmentation FAERS data does not arrive as a single clean table. Each quarterly release contains seven separate ASCII files that need to be joined on a shared identifier. A single adverse event case is split across demographics, drug records, reaction terms, and outcome codes, each in a different file. On top of that, the same case can appear multiple times across quarters as manufacturers and healthcare providers submit follow-up updates. Without careful deduplication, the same event gets counted multiple times and inflates signal counts. Drug names add another layer of complexity: the same active ingredient can appear under dozens of brand names, generic names, and free-text variations, making it impossible to aggregate signals reliably without normalization.

Manual Workflows Today, pharmacovigilance analysts spend significant time manually querying FAERS, searching PubMed for supporting literature, and writing safety narratives. These tasks are repetitive, time-consuming, and bottlenecked by the availability of trained personnel. The FDA itself publishes quarterly FAERS signal reports, but these are produced after internal review cycles that can take months. By the time a signal is formally communicated, it has often been visible in the data for considerably longer.

Lack of Intelligent Automation Existing signal detection tools largely stop at the statistical layer. They compute PRR or similar metrics and present a list of elevated drug-reaction pairs, but they do not automatically retrieve supporting literature, assess outcome severity, or generate a human-readable summary. The analyst still has to do the synthesis work manually, which limits how quickly a signal can move from detection to review.

Big Data Bottlenecks FAERS receives over two million reports per year. A single quarterly file contains roughly 400,000 to 500,000 cases, with each case producing multiple drug-reaction pair combinations after joining. Computing PRR across all possible pairs in a full year of data means aggregating tens of millions of rows. This is not feasible with standard single-machine tooling at production scale, and it gets more expensive every quarter as the database grows.

4.2 Opportunities

Scalable Pipelines By ingesting FAERS data through Kafka and processing it with Spark, MedSignal can handle the full quarterly data volume in a matter of minutes rather than hours. The pipeline is designed to run again each time a new quarter is released, with the same code and no manual intervention. As data volume grows across years, the architecture scales horizontally without requiring a redesign.

LLM-Assisted Analysis Rather than stopping at a PRR number, MedSignal uses a three-agent LLM pipeline to automatically retrieve relevant PubMed literature, score its relevance to the detected signal, and generate a structured SafetyBrief that contextualizes the statistical finding with published evidence. This compresses what would typically be hours of analyst work into a process that completes in seconds per signal.

Automated Signal Prioritization The Signal Severity Score gives analysts a single ranked view of all flagged signals ordered by statistical strength and literature support. Instead of reviewing hundreds of PRR values manually, the analyst sees a prioritized queue where the most concerning signals surface first. Every signal comes with a pre-generated SafetyBrief, cited PMIDs, and outcome severity indicators, so the reviewer has everything they need in one place.

Earlier Detection Our preliminary proof-of-concept analysis on 2023 FAERS data identified candidate signals for ten drug-reaction pairs with documented FDA communications in 2023. The POC suggests that several of these signals were statistically detectable in FAERS data weeks to months before the FDA formally communicated them. The full validated pipeline will quantify this detection lead time for each golden drug. If confirmed, this demonstrates that automated PRR-based signal detection on routinely available FAERS data could meaningfully accelerate the pharmacovigilance review cycle.

Methodology

5.1 Data Sources

FDA FAERS 2023 (Primary Source) The core dataset is FDA FAERS quarterly data for all four quarters of 2023, downloaded directly from the FDA public portal at

https://fis.fda.gov/extensions/FPD-QDE-FAERS/FPD-QDE-FAERS.html. Each quarterly ZIP contains seven ASCII files delimited by the dollar sign character. We use four of these files.

File	Rows (Q1 2023)	What it contains
DEMO	432,144	Patient demographics, report dates, reporter details
DRUG	1,899,503	Drug records per case including role and active ingredient
REAC	1,491,473	MedDRA adverse reaction terms per case
OUTC	309,217	Outcome codes including death, hospitalization, life-threatening

Across all four quarters of 2023, the stacked raw dataset contains 1,673,637 DEMO rows and 7,473,722 DRUG rows. After caseversion deduplication, Primary Suspect filtering, three-file join, and pair-level deduplication, the clean dataset contains **1,544,430 unique patient cases** and **5,012,904 drug-reaction rows** spanning 6,425 unique drugs and 16,381 unique MedDRA reaction terms. These figures are confirmed from our full-year EDA run.

The three files we are not using are THER (therapy dates), INDI (drug indications), and RPSR (report source). These were excluded because THER and INDI have significant missingness that makes them unreliable for our analysis, and RPSR does not contribute to PRR computation or signal severity scoring.

Expected Data Volume Raw ASCII files across four quarters total approximately 2 to 3 GB uncompressed. After filtering and joining, the drug_reaction_pairs table in PostgreSQL is expected to hold 6 to 8 million rows at roughly 1 to 2 GB of indexed storage.

PubMed via NCBI Entrez API (Secondary Source) We fetch PubMed abstracts for each of our ten golden drug-reaction pairs using targeted boolean search queries through the NCBI Entrez API. Each query combines the drug name with the specific adverse reaction term and publication type filters to focus on clinical trials, case reports, and review articles. We expect to retrieve 150 to 200 relevant abstracts per drug-reaction pair, giving a total corpus of approximately 1,500 to 2,000 abstracts. These are embedded once using the all-MiniLM-L6-v2 sentence transformer model running locally and stored in ChromaDB. No API key is required for NCBI at our usage volume, though we will register an email address to increase the rate limit from 3 to 10 requests per second.

NIH RxNorm API (Supporting Source) The NIH RxNorm REST API is used once during the preparation phase to resolve drug names from the prod_ai field to canonical RxCUI identifiers. Results are cached in PostgreSQL before the pipeline runs so that no live API calls are made during Spark processing. No API key is required.

5.2 Technology Stack

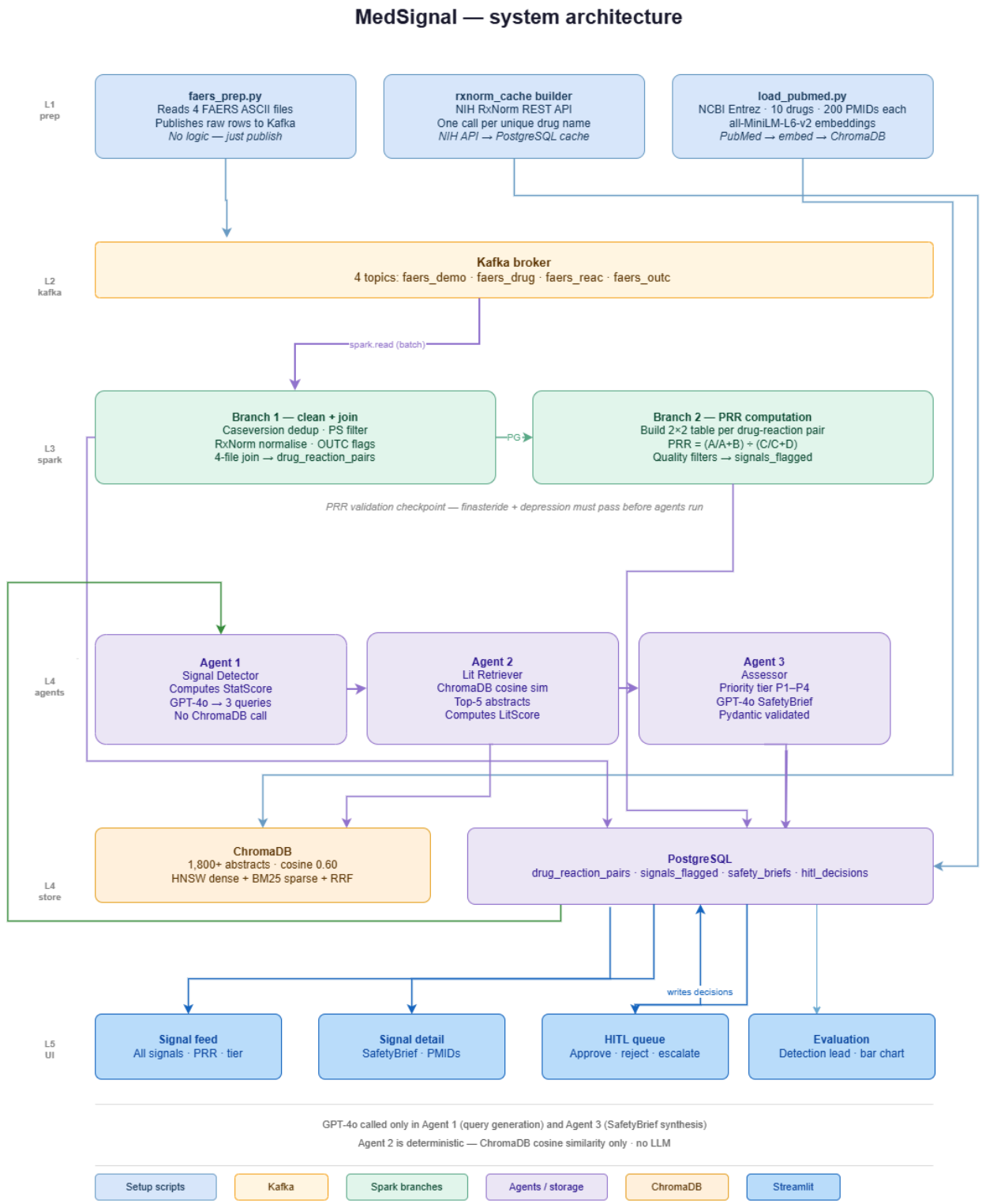
Component	Technology	Justification
Data ingestion	Apache Kafka	Decouples FAERS file preparation from Spark processing. Enables future extension to real-time adverse event feeds without changing downstream components.
Stream processing	Apache Spark Structured Streaming	Handles parallel normalization and PRR aggregation across 6 to 8 million drug-reaction pairs. Scales horizontally for future quarterly data additions.
Relational storage	PostgreSQL	Stores drug-reaction pairs, flagged signals, safety briefs, and HITL decisions. Indexed schemas support sub-second query performance at our data volume.
Vector store	ChromaDB	Local persistent store for PubMed embeddings. No external service dependency, no API cost, and retrieval latency is acceptable at our corpus size of 1,500 to 2,000 abstracts.
Embeddings	all-MiniLM-L6-v2 (HuggingFace)	Runs locally with no API cost. 384-dimensional embeddings are well suited for semantic similarity search over biomedical abstracts at our scale.
Agent framework	LangGraph	Provides a clean stateful pipeline for our three-agent workflow with conditional routing and explicit state management between agents.

LLM	GPT-4o mini (OpenAI)	Used for SafetyBrief generation in Agent 3. Cost-effective for our usage volume. Can be swapped for any OpenAI-compatible model without changing agent code.
Drug normalization	NIH RxNorm API	Resolves prod_ai drug names to canonical RxCUI identifiers. Called once during preparation and cached in PostgreSQL.
Output validation	Pydantic v2	Enforces SafetyBrief schema before writing to PostgreSQL. Catches malformed LLM outputs before they reach the HITL queue.
Frontend	Streamlit	Rapid development of a multi-page analyst interface. Connects directly to PostgreSQL with no API layer required.
Containerization	Docker Compose	Runs all components including Kafka, Spark, PostgreSQL, and ChromaDB in a consistent environment across all team members.
Observability	Prometheus	Scrapes pipeline metrics including messages processed, Spark job duration, signals flagged, and HITL queue depth.

5.3 Architecture

System Overview

MedSignal is organized into five layers that each have a distinct responsibility. Data moves top to bottom through the layers, with no layer reaching back up to query a layer above it. Figure below shows the full system architecture diagram.



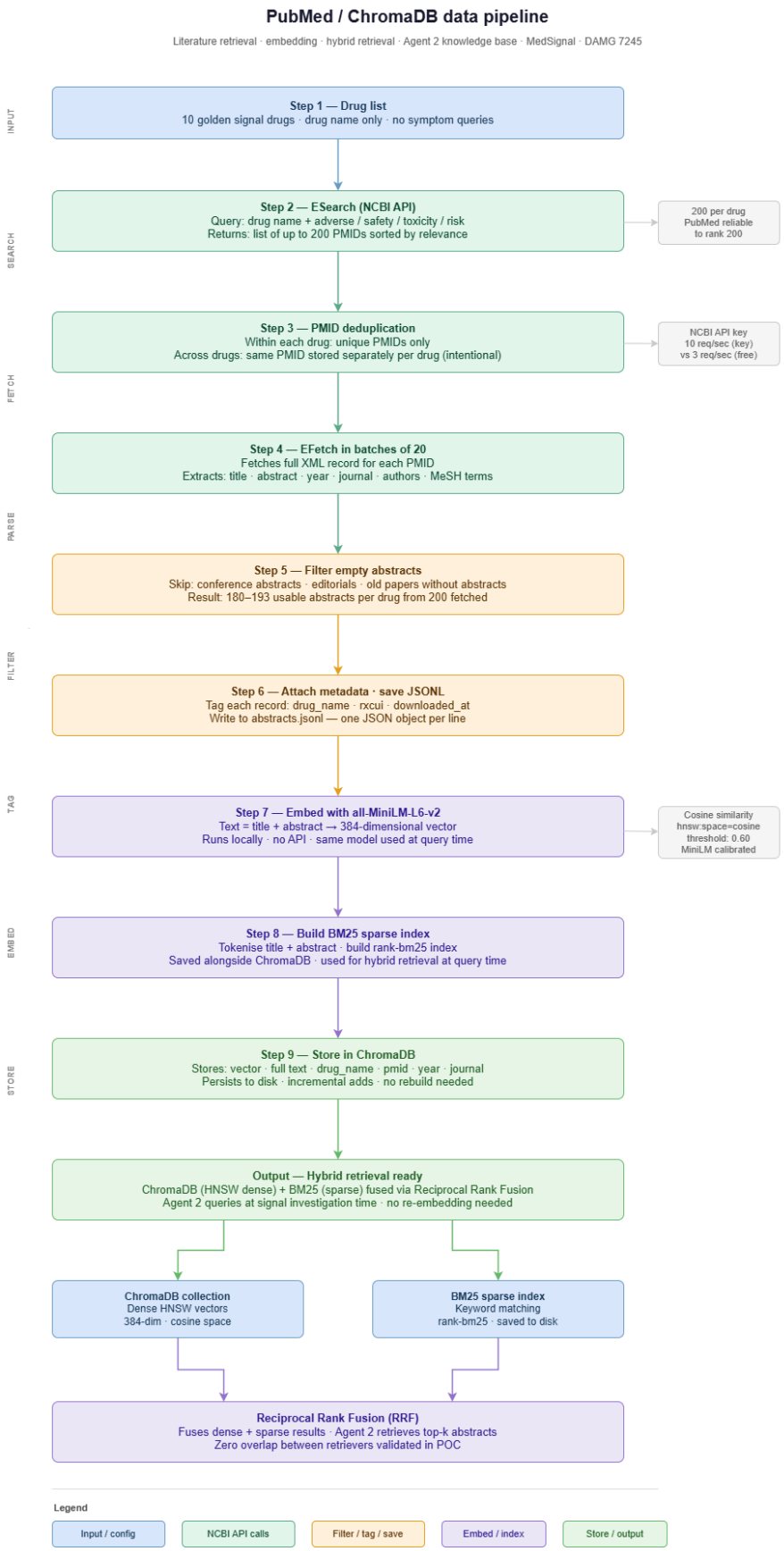
Layer 1: Data Preparation

Three one-time setup scripts prepare the storage layer before the pipeline runs.

faers_prep.py reads the four raw FAERS ASCII files for all four quarters of 2023 and publishes the raw records as-is to four separate Kafka topics: faers_demo, faers_drug, faers_reac, and faers_outc. No joining, filtering, normalization, or deduplication takes place in this script. All of that processing happens in Spark.

rxnorm_cache builder calls the NIH RxNorm REST API once for each unique drug name found in the raw DRUG files and stores the resolved canonical names in a PostgreSQL cache table. This ensures no live API calls are made during Spark processing. Normalization in Branch 1 is a fast local lookup against this pre-built cache.

load_pubmed.py fetches PubMed abstracts for each of the ten golden signal drugs using targeted NCBI Entrez queries. Each query combines the drug name with adverse, safety, toxicity, and risk terms to retrieve up to 200 PMIDs sorted by relevance. After filtering out conference abstracts, editorials, and records with empty abstracts, approximately 180 to 193 usable abstracts remain per drug, giving a total corpus of approximately 1,800 to 1,930 abstracts. Each record is tagged with drug_name, rxcui, and downloaded_at metadata before being embedded using all-MiniLM-L6-v2 locally and stored in ChromaDB. The collection uses cosine similarity with a threshold of 0.60 for retrieval.

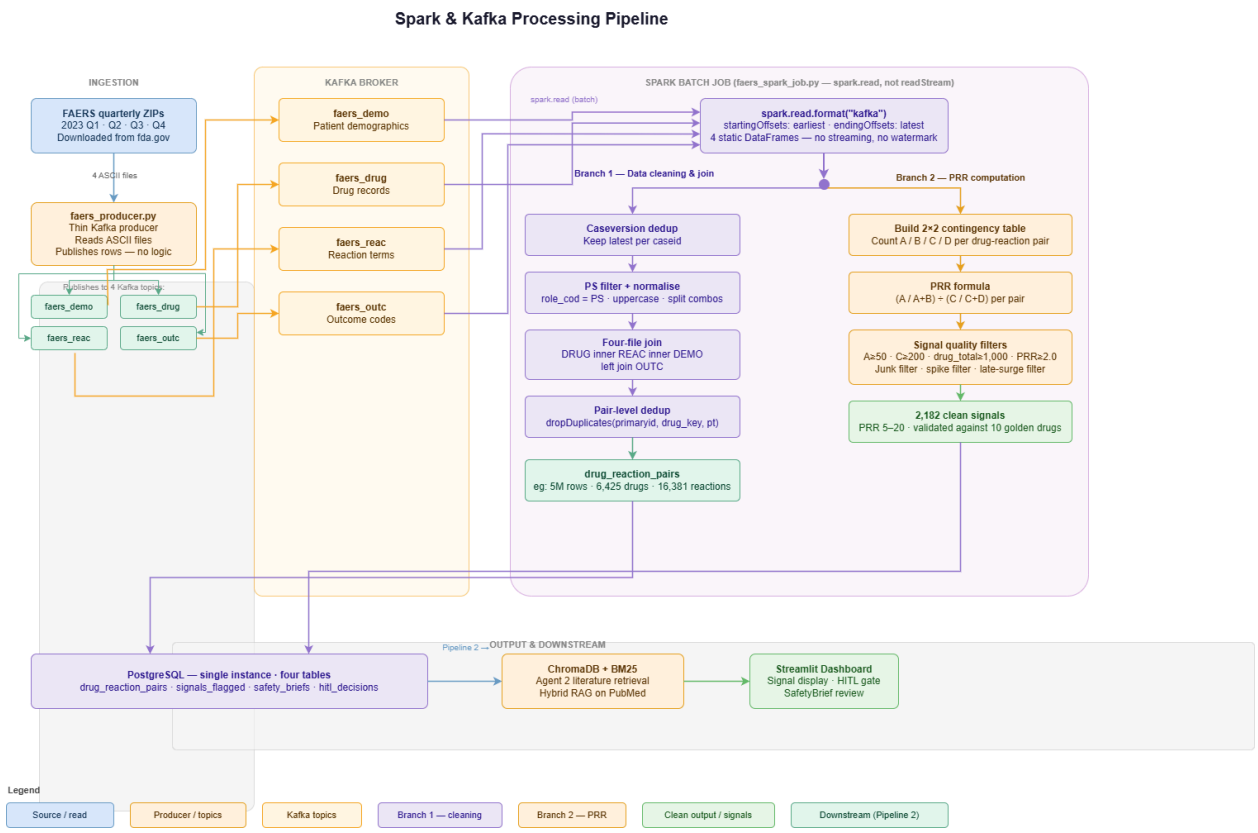

PubMed / ChromaDB data pipeline
Literature retrieval · embedding · hybrid retrieval · Agent 2 knowledge base · MedSignal · DAMG 7245
INPUT
Step 1 — Drug list
10 golden signal drugs · drug name only · no symptom queries
SEARCH
Step 2 — ESearch (NCBI API)
Query: drug name + adverse / safety / toxicity / risk
Returns: list of up to 200 PMIDs sorted by relevance
200 per drug
PubMed reliable
to rank 200
FETCH
Step 3 — PMID deduplication
Within each drug: unique PMIDs only
Across drugs: same PMID stored separately per drug (intentional)
NCBI API key
10 req/sec (key)
vs 3 req/sec (free)
Step 4 — EFetch in batches of 20
Fetches full XML record for each PMID
Extracts: title · abstract · year · journal · authors · MeSH terms
PARSE
Step 5 — Filter empty abstracts
Skip: conference abstracts · editorials · old papers without abstracts
Result: 180–193 usable abstracts per drug from 200 fetched
FILTER
Step 6 — Attach metadata · save JSONL
Tag each record: drug_name · rxcui · downloaded_at
Write to abstracts.jsonl — one JSON object per line
TAG
Step 7 — Embed with all-MiniLM-L6-v2
Text = title + abstract → 384-dimensional vector
Runs locally · no API · same model used at query time
Cosine similarity
hnsw:space=cosine
threshold: 0.60
MiniLM calibrated
EMBED
Step 8 — Build BM25 sparse index
Tokenise title + abstract · build rank-bm25 index
Saved alongside ChromaDB · used for hybrid retrieval at query time
STORE
Step 9 — Store in ChromaDB
Stores: vector · full text · drug_name · pmid · year · journal
Persists to disk · incremental adds · no rebuild needed
Output — Hybrid retrieval ready
ChromaDB (HNSW dense) + BM25 (sparse) fused via Reciprocal Rank Fusion
Agent 2 queries at signal investigation time · no re-embedding needed
ChromaDB collection
Dense HNSW vectors
384-dim · cosine space
BM25 sparse index
Keyword matching
rank-bm25 · saved to disk
Reciprocal Rank Fusion (RRF)
Fuses dense + sparse results · Agent 2 retrieves top-k abstracts
Zero overlap between retrievers validated in POC
Legend
Input / config
NCBI API calls
Filter / tag / save
Embed / index
Store / output

Layer 2: Kafka Ingestion

Kafka acts as a durable buffer between file ingestion and Spark processing. All four topics are populated in a single run of faers_prep.py. When a new quarter arrives, the script runs again for that quarter only and Spark picks up the new records using Kafka offsets without reprocessing earlier quarters.

Layer 3: Spark Processing (Batch Mode)

Spark reads all four Kafka topics as static dataframes using spark.read in batch mode, not spark.readStream. This is the correct API choice because FAERS data is a fixed quarterly dataset. All records exist at the moment Spark runs. There is no late-arriving data, no watermarking, and no streaming state management.



Branch 1 — Join, Dedup, Normalize reads all four topics as static dataframes and performs the full data engineering pipeline:

Step 1: Deduplicate DEMO — keep highest caseversion per caseid

Step 2: Filter DRUG to role_cod = PS only

Step 3: Normalize drug names via RxNorm cache broadcast join

Fallback: prod_ai → raw drugname

Step 4: Aggregate OUTC flags per primaryid

(death_flag, hosp_flag, lt_flag)

Step 5: Join all four on primaryid

Step 6: Write drug_reaction_pairs to PostgreSQL

Branch 2 — PRR Computation reads drug_reaction_pairs from PostgreSQL once Branch 1 completes and computes the Proportional Reporting Ratio across all drug-reaction combinations. The PRR formula is:

PRR = (A / (A + B)) / (C / (C + D))

Where:

A = cases reporting drug X with reaction Y

B = cases reporting drug X without reaction Y

C = cases reporting any other drug with reaction Y

D = cases reporting any other drug without reaction Y

Drug-reaction pairs that pass the following filters are written to the signals_flagged table in PostgreSQL. Current thresholds based on preliminary POC analysis are A >= 50, C >= 200, drug_total >= 1,000, and PRR >= 2.0. These values may be adjusted during full pipeline validation.

The Spark job runs once after all quarterly records have been consumed from Kafka. Results are validated against the known finasteride-depression PRR benchmark before the agent pipeline is triggered.

Layer 4: LangGraph Agent Pipeline

The agent pipeline is triggered once after Spark completes and signals_flagged is populated in PostgreSQL. Three agents execute in sequence with shared state passed between them via LangGraph.

Agent 1 (Signal Detector) reads each flagged signal from PostgreSQL and computes a StatScore from three components:

prr_score = min(PRR / 4.0, 1.0)

volume_score = min(log10(case_count) / log10(50), 1.0)

severity_score = 1.0 if death · 0.75 if life-threatening
0.50 if hospitalization · 0.0 otherwise

StatScore = (prr_score × 0.50) +

(volume_score × 0.30) +

(severity_score × 0.20)

Agent 1 then calls GPT-4o to generate three natural language PubMed search queries tailored to the specific drug-reaction signal. A template-based approach cannot do this effectively — GPT-4o's biomedical knowledge generates domain-specific terminology that makes ChromaDB retrieval significantly more effective. For example, for dupilumab and conjunctivitis, GPT-4o produces queries such as "dupilumab conjunctivitis ocular adverse effects mechanism", "dupilumab eye inflammation incidence epidemiology", and "dupilumab conjunctivitis clinical outcomes risk factors." These three queries are passed to Agent 2.

Agent 2 (Literature Retriever) queries ChromaDB using the drug name and reaction term as a combined search string, filtered to the relevant pair_key. It retrieves the top five most relevant abstracts and computes a LitScore:

relevance = 1.0 - avg(distances) / 1.5

volume = min(len(docs) / 5.0, 1.0)

LitScore = (relevance × 0.70) + (volume × 0.30)

If no abstracts are found, LitScore = 0.0. No LLM is used. ChromaDB semantic search handles retrieval entirely.

Agent 3 (Assessor) takes both scores, assigns a priority tier, and calls the LLM to generate a SafetyBrief:

P1: StatScore >= 0.7 AND LitScore >= 0.5 (review first)

P2: StatScore >= 0.7 AND LitScore < 0.5 (review second)

P3: StatScore < 0.7 AND LitScore >= 0.5 (review third)

P4: StatScore < 0.7 AND LitScore < 0.5 (review last)

The LLM prompt includes the drug, reaction, PRR, outcome counts, retrieved abstracts, and PMIDs. The response is validated by Pydantic against the SafetyBrief schema before being written to PostgreSQL. The LLM is used only here because StatScore and LitScore are deterministic computations. The synthesis task combining statistical evidence with published literature into a coherent clinical narrative — is where natural language generation genuinely adds value.

We present StatScore and LitScore independently rather than combining them into a single weighted score. Assigning weights to statistical versus literature evidence requires pharmacovigilance domain expertise we do not have. The reviewer sees both scores transparently and makes the final judgment.

Layer 5: Streamlit Interface

The analyst interface connects directly to PostgreSQL and presents four pages.

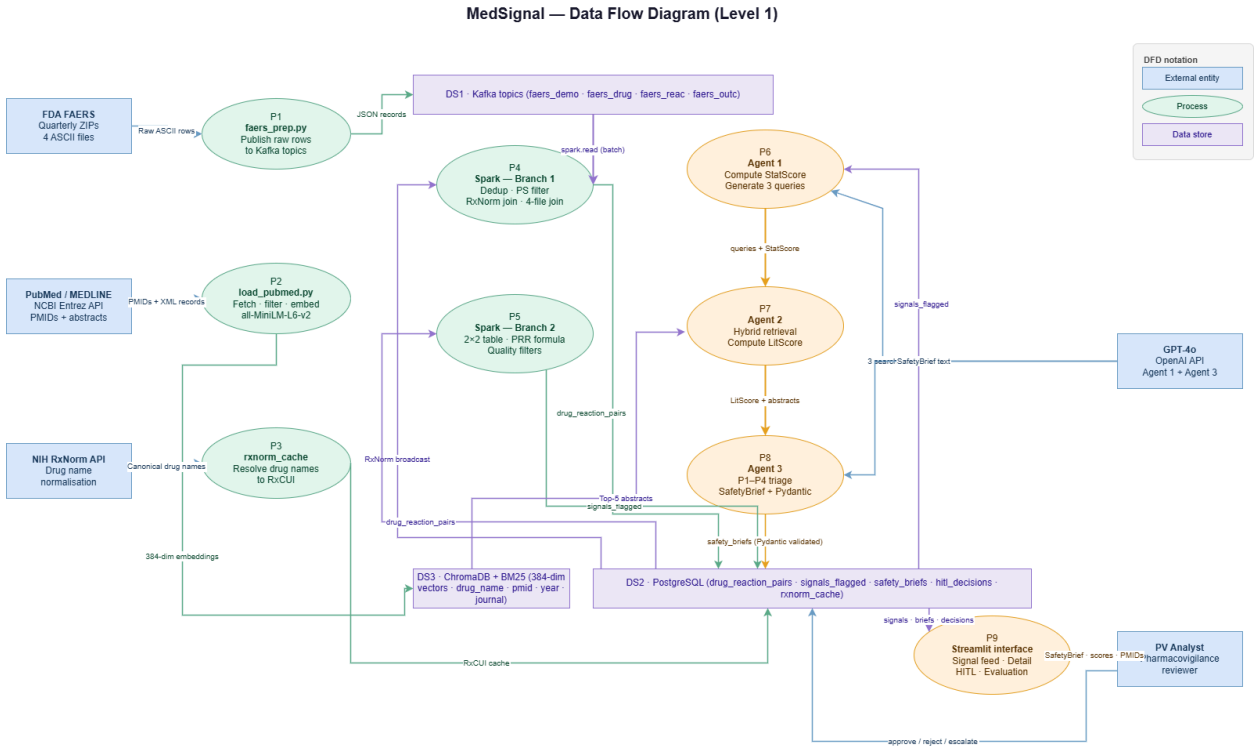
Signal Feed displays all entries in signals_flagged ranked by priority tier, with PRR, StatScore, LitScore, case count, and outcome severity visible at a glance.

Signal Detail shows the full SafetyBrief for a selected signal, the StatScore and LitScore with their component breakdowns, and cited PMIDs with abstract previews.

HITL Queue presents every signal for human review sorted by priority tier so P1 signals appear first. The analyst reads the SafetyBrief and makes an approve, reject, or escalate decision. Each decision is written to the hitl_decisions table with a timestamp.

Evaluation Dashboard shows the detection lead time for each of the ten golden drugs as a bar chart and a precision-recall table showing how many golden signals the system correctly flagged above the PRR threshold.

Below is the Data Flow Diagram-



5.4 Data Processing and Transformation

Batch vs stream processing

MedSignal uses batch processing throughout. FAERS data arrives as quarterly bulk files — there is no continuous stream of incoming records. The Kafka layer decouples file ingestion from Spark computation, but Spark reads all Kafka topics using spark.read in batch mode, not spark.readStream. This is the correct design choice for a quarterly refresh data source. The same pipeline re-runs when new quarterly data is released, with Spark picking up only new records via Kafka offsets.

Data formats

All four FAERS source files are ASCII text, dollar-sign delimited, with latin1 encoding. This was confirmed from direct inspection of the 2023 quarterly files. Records are published to Kafka as JSON messages and deserialized within Spark using schema-enforced from_json calls. Final outputs are written to PostgreSQL in relational table format and to ChromaDB as 384-dimensional dense vectors.

Storage schemas

Four PostgreSQL tables hold all relational outputs.

drug_reaction_pairs is the output of Spark Branch 1. One row per unique primaryid, drug_key, and pt combination after all joins, deduplication, and filtering. Key columns are primaryid, caseid, drug_key, pt, fda_dt, death_flag, hosp_flag, lt_flag, and source_quarter. This table holds approximately 5 million rows across all four 2023 quarters.

signals_flagged is the output of Spark Branch 2. One row per drug_key and pt combination that cleared all PRR thresholds and quality filters. Key columns are drug_key, pt, PRR, case_count, death_count, hosp_count, StatScore, and computed_at.

safety_briefs is the output of the LangGraph agent pipeline. One row per signal containing StatScore, LitScore, priority tier, SafetyBrief text, cited PMIDs, and the model used to generate the brief.

hitl_decisions is the output of the Streamlit HITL queue. One row per reviewer decision containing the signal reference, decision value (approve, reject, or escalate), reviewer note, and timestamp.

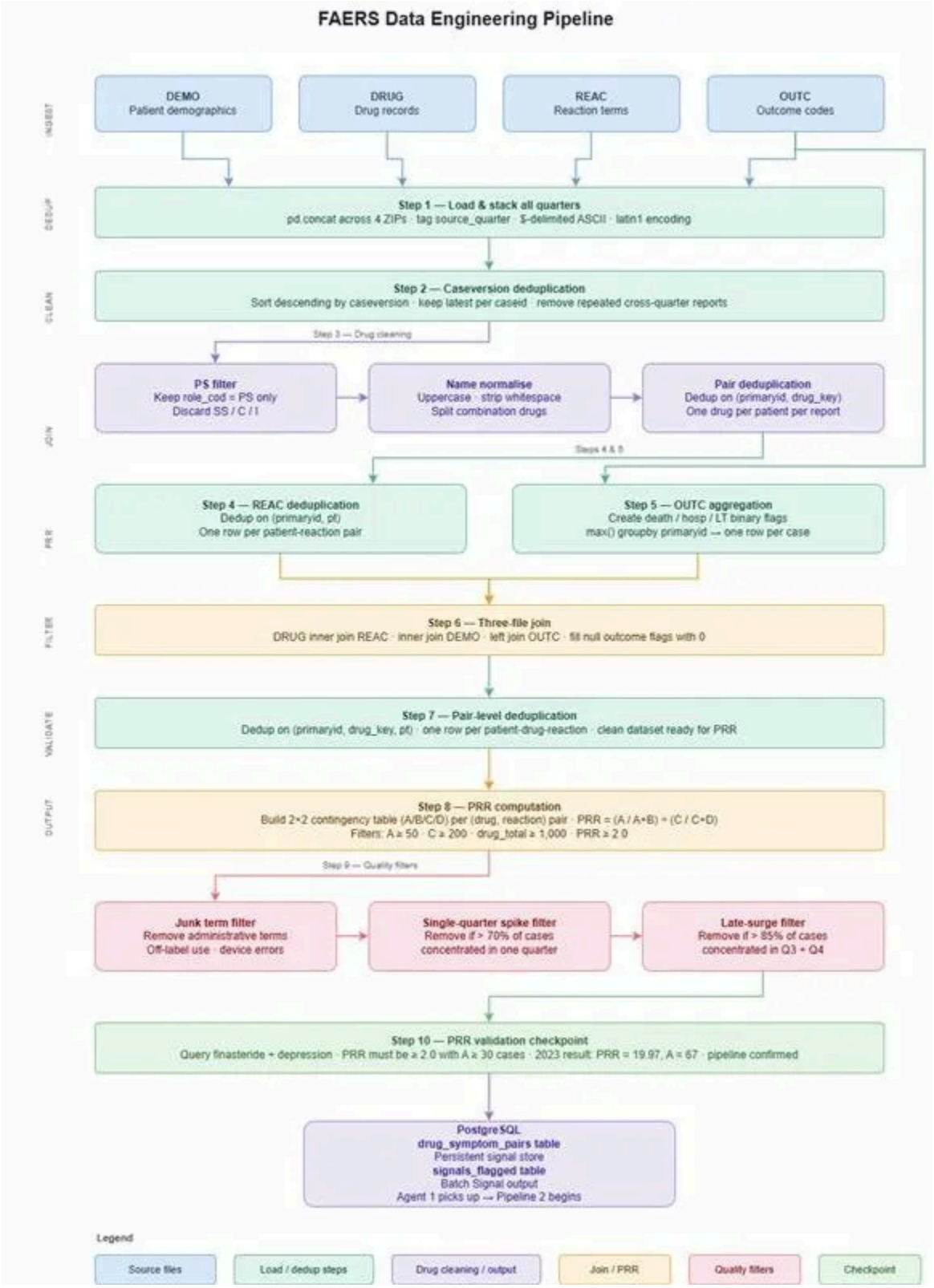
Parallel processing strategy

Spark processes Branch 1 and Branch 2 sequentially within a single batch job. Branch 1 performs the data engineering work in parallel across Spark partitions — the four-file join, deduplication, PS filtering, and RxNorm normalization are all distributed operations. Branch 2 reads the completed drug_reaction_pairs table and computes PRR aggregations using Spark's groupBy, which parallelizes across partitions automatically.

The RxNorm cache is loaded as a broadcast variable before the join so that every Spark executor has a local copy, avoiding repeated PostgreSQL lookups during normalization.

Data transformations in Branch 1

The full transformation sequence applied to the raw FAERS records is as follows.



Step 1 — Load and stack all quarters. All four 2023 quarterly files are loaded per file type using pd.concat, tagged with a source_quarter column, and published to Kafka as raw records. Dollar-sign delimiter and latin1 encoding are applied at load time.

Step 2 — Caseversion deduplication. Within the DEMO stream, records are sorted descending by caseversion and deduplicated on caseid using a Spark window function, keeping the latest version of each case. This removes repeated cross-quarter follow-up reports from the case count.

Step 3 — PS filter and drug name normalization. The DRUG stream is filtered to role_cod = PS only, discarding secondary suspect, concomitant, and interacting drug records. Drug names are normalized through a three-level hierarchy: prod_ai active ingredient field as the primary source, RxNorm canonical name from the pre-built cache as the fallback when prod_ai is missing or unresolvable, and raw drugname as a last resort. Combination drugs indicated by a backslash separator are treated as a single entity.

Step 4 — REAC deduplication. Within each case, duplicate reaction terms are removed on the combination of primaryid and pt to ensure one row per patient-reaction pair.

Step 5 — OUTC aggregation. Outcome codes are aggregated per primaryid using max() to produce three binary flags per case: death_flag for outcome code DE, hosp_flag for HO, and lt_flag for LT.

Step 6 — Four-file join. The deduplicated DEMO, filtered and normalized DRUG, and deduplicated REAC dataframes are joined using inner joins on primaryid. OUTC is joined using a left join to preserve cases with no recorded outcome. Null outcome flags are filled with zero.

Step 7 — Pair-level deduplication. The joined dataset is deduplicated on the combination of primaryid, drug_key, and pt to produce one clean row per patient-drug-reaction triple. This is the drug_reaction_pairs table written to PostgreSQL.

Data transformations in Branch 2

Branch 2 reads drug_reaction_pairs from PostgreSQL and applies the following transformations.

Step 8 — PRR computation. A 2x2 contingency table is built per drug-reaction pair. The PRR formula is PRR = (A / A+B) divided by (C / C+D), where A is cases reporting drug X with reaction Y, B is cases reporting drug X without reaction Y, C is cases reporting any other drug with reaction Y, and D is cases reporting any other drug without reaction Y.

Step 9 — Threshold filtering. Pairs are filtered to A >= 50, C >= 200, drug_total >= 1,000, and PRR >= 2.0.

Step 10 — Quality filters. Three additional filters are applied sequentially. The junk term filter removes administrative MedDRA terms including drug ineffective, product use issue, and off-label use. The single-quarter spike filter removes signals where more than 70% of cases are concentrated in a single quarter, catching litigation-driven reporting spikes. The late-surge filter

removes signals where more than 85% of cases appear in Q3 and Q4 only, catching signals driven by media attention rather than organic accumulation.

Step 11 — PRR validation checkpoint. Before writing to signals_flagged, a validation query confirms that finasteride and depression appears in the output with PRR meeting the threshold. This is the pipeline correctness gate — if it fails, the job stops and the issue is investigated before any downstream components are triggered.

Embedding generation

PubMed abstracts are embedded outside the Spark pipeline using the all-MiniLM-L6-v2 sentence transformer model running locally via HuggingFace. The embedding input is the concatenation of the abstract title and body text, producing a 384-dimensional dense vector per record. The model is the same at both index time and query time, ensuring consistent vector space alignment between stored embeddings and Agent 1's search queries. Embeddings are stored in ChromaDB with drug_name, pmid, year, and journal metadata fields to support filtered retrieval.

5.5 LLM Integration Strategy

Overview

MedSignal uses GPT-4o at two specific points in the agent pipeline where natural language generation or domain-specific reasoning genuinely adds value. GPT-4o is not used for statistical computation, data retrieval, or scoring — those tasks are handled deterministically. This disciplined approach keeps token costs predictable, makes outputs reproducible, and ensures LLM usage can be validated against ground truth.

Prompt Design

Agent 1 — Search Query Generation

Agent 1 calls GPT-4o to generate three natural language PubMed search queries for the flagged drug-reaction signal. A template approach cannot do this effectively because biomedical retrieval quality depends on domain-specific terminology that varies by drug class, mechanism of action, and reaction type. GPT-4o's pretraining on biomedical literature allows it to generate queries that capture clinical context beyond simple keyword matching.

The prompt is structured as follows:

System:
You are a pharmacovigilance expert. Given a drug name and adverse
reaction, generate exactly 3 PubMed search queries that would
retrieve the most relevant clinical and safety literature.
Each query should approach the topic from a different angle:
mechanistic, epidemiological, and clinical outcomes.

Return only the 3 queries as a JSON array. No explanation.

User:
Drug: {drug_key}
Reaction: {pt}
PRR: {prr} based on {case_count} cases

Example output for dupilumab and conjunctivitis:

```
json
[
  "dupilumab conjunctivitis ocular adverse effects mechanism",
  "dupilumab eye inflammation incidence epidemiology",
  "dupilumab conjunctivitis clinical outcomes risk factors"
]
```

The three queries are passed directly to Agent 2 for ChromaDB retrieval.

Agent 3 — SafetyBrief Generation

Agent 3 calls GPT-4o to synthesize the statistical evidence and retrieved literature into a structured SafetyBrief. This is the task that genuinely requires a language model combining a PRR value, outcome counts, and five PubMed abstracts into a coherent clinical narrative that a pharmacovigilance analyst can act on.

The prompt is structured as follows:

System:
You are a pharmacovigilance medical writer. Write a concise safety
brief for a drug-reaction signal. Be factual, cite PMIDs inline,
and do not introduce claims not supported by the provided evidence.
Return a JSON object matching the schema exactly. No extra text.

User:
Drug: {drug_key}
Reaction: {pt}
PRR: {prr} | Cases: {case_count}
Deaths: {death_count} | Hospitalizations: {hosp_count}
StatScore: {stat_score} | LitScore: {lit_score}
Priority tier: {priority}

Retrieved abstracts:
{abstracts_with_pmids}

Return JSON:
{
  "brief_text": "...",

```
  "key_findings": ["...", "..."],
  "pmids_cited": ["...", "..."],
  "recommended_action": "..."
}
```

Output is validated by Pydantic v2 before being written to PostgreSQL. If validation fails, the agent retries once with a stricter prompt instructing the model to return only valid JSON.

Retrieval-Augmented Generation

Agent 2 handles retrieval without an LLM. It embeds the three search queries from Agent 1 using all-MiniLM-L6-v2 — the same model used at index time and runs cosine similarity search against ChromaDB filtered by drug_name metadata. Only abstracts with a cosine similarity score above 0.60 are returned. Up to five abstracts are passed to Agent 3.

The RAG design deliberately separates retrieval from generation. Agent 2 finds the evidence. Agent 3 synthesizes it. This separation makes it possible to evaluate each step independently retrieval quality can be assessed by checking whether returned abstracts are actually relevant to the signal, and generation quality can be assessed by checking whether the SafetyBrief accurately reflects what the abstracts say.

Agentic Workflow

The three agents form a linear LangGraph state machine. State is passed between agents as a typed Python dictionary. There are no loops, no tool calls, and no autonomous decision-making beyond priority tier assignment. The pipeline is deterministic in structure — it always runs Agent 1, then Agent 2, then Agent 3, in that order.

```
State schema:
  drug_key:        str
  pt:              str
  prr:             float
  case_count:      int
  death_count:     int
  hosp_count:      int
  stat_score:      float
  search_queries:  List[str]     ← added by Agent 1
  abstracts:       List[dict]    ← added by Agent 2
  lit_score:       float         ← added by Agent 2
  priority:        str           ← added by Agent 3
  brief:           SafetyBrief   ← added by Agent 3
```

The pipeline is triggered once per signal in signals_flagged. For ten golden drugs the full pipeline completes in under two minutes.

API Usage Pattern

Both GPT-4o calls use the OpenAI chat completions API with temperature set to 0 for reproducibility. Agent 1 uses a max token limit of 200 since it only needs three short queries. Agent 3 uses a max token limit of 600 to allow a complete SafetyBrief with citations.

A hard spend limit is set on the OpenAI account before any pipeline run. During development, GPT-4o mini is used in place of GPT-4o to reduce cost. The final evaluation and demo run use GPT-4o for output quality.

How the LLM Contributes to the Problem

Without the LLM layer, MedSignal would produce a ranked list of PRR values — statistically meaningful but not clinically actionable. A pharmacovigilance analyst reviewing hundreds of flagged signals needs more than a number. They need to know what the literature says, what the mechanism might be, how serious the outcomes are, and what action is recommended.

GPT-4o contributes at two points. In Agent 1 it improves retrieval quality by generating domain-aware search queries. In Agent 3 it compresses evidence synthesis from hours of manual literature review into seconds of automated generation. Both uses are grounded in retrieved evidence and validated against a defined output schema, which limits hallucination risk and makes the outputs auditable.

5.6 Guardrails and Human-in-the-Loop (HITL)

Overview

MedSignal operates in a safety-critical domain where incorrect outputs could influence decisions about drug safety. We apply guardrails at three points: before the LLM is called, on the LLM output before it is stored, and at the human review stage before any signal is acted upon. Every flagged signal passes through human review regardless of its score. There is no automated approval or rejection.

Input Guardrails

Statistical threshold gate Before any signal reaches the agent pipeline, it must have cleared the PRR threshold filters in Spark Branch 2 — A >= 50, C >= 200, drug_total >= 1,000, and PRR >= 2.0 — plus all three quality filters. This ensures the LLM never receives a signal that is statistically weak or driven by data artefacts such as litigation spikes or late-quarter surges.

PRR validation checkpoint Before the agent pipeline is triggered, a validation query confirms that the finasteride-depression signal appears in signals_flagged with a passing PRR. If this checkpoint fails, the pipeline stops. No agents run until the data layer is confirmed correct.

Prompt constraints The Agent 3 system prompt explicitly instructs GPT-4o not to introduce claims beyond the provided evidence. The instruction "do not introduce claims not supported by the provided evidence" is included in every Agent 3 call. This is a soft guardrail; it does not prevent hallucination but reduces it and makes the model's task scope explicit.

Output Validation

Pydantic v2 schema enforcement Every Agent 3 response is validated against a strict Pydantic schema before being written to PostgreSQL. The schema enforces field types, required fields, and value constraints:

```python
class SafetyBrief(BaseModel):
    brief_text:        str
    key_findings:      List[str]
    pmids_cited:       List[str]
    recommended_action: str
    drug_key:          str
    pt:                str
    stat_score:        float = Field(ge=0.0, le=1.0)
    lit_score:         float = Field(ge=0.0, le=1.0)
    priority:          Literal["P1", "P2", "P3", "P4"]
    generated_at:      datetime
```

If validation fails on the first attempt, Agent 3 retries once with a stricter prompt that includes the schema definition and the specific validation error. If the second attempt also fails, the signal is flagged with a generation_error status in PostgreSQL and routed to HITL with a note indicating the brief could not be generated. The reviewer sees the raw StatScore, LitScore, PRR, and outcome data and makes their decision without a SafetyBrief.

PMID citation check After SafetyBrief generation, a post-processing step verifies that every PMID cited in brief_text appears in the pmids_cited list returned by Agent 2. Any PMID cited in the text but not in the retrieved set is flagged as a potential hallucination and removed from the brief before it is stored. This prevents the model from citing papers it invented rather than papers it was given.

Human-in-the-Loop Design

All signals are routed to HITL Every signal that clears the PRR thresholds and passes quality filters is placed in the HITL review queue. There is no automated approval path. This design reflects the safety-critical nature of pharmacovigilance automated scoring should inform human judgment, not replace it.

Priority-based queue ordering Signals are sorted in the HITL queue by priority tier so reviewers see the most important signals first:

P1: StatScore >= 0.7 AND LitScore >= 0.5 (review first)
P2: StatScore >= 0.7 AND LitScore < 0.5 (review second)
P3: StatScore < 0.7 AND LitScore >= 0.5 (review third)
P4: StatScore < 0.7 AND LitScore < 0.5 (review last)

Within each tier, signals are further sorted by StatScore descending.

Reviewer interface The Streamlit HITL page presents each signal with the full SafetyBrief, StatScore and LitScore values, PRR, case count, death and hospitalization counts, and the retrieved PubMed abstracts with clickable PMID links. The reviewer selects one of three decisions:

Approve — signal is confirmed as valid, logged for reporting
Reject — signal is dismissed, reason recorded
Escalate — signal requires further expert review, flagged separately

An optional free-text note field is available for all three decisions.

Immutable decision log Every HITL decision is written as a new row in the hitl_decisions table with a timestamp. Decisions are never updated in place — if a reviewer changes their mind, a new row is inserted. This gives a complete audit trail of every review event, which is standard practice in regulated pharmacovigilance environments.

What is Not Automated

To be explicit about the boundaries of the system, the following decisions are never made automatically by MedSignal:

- Whether a signal represents a genuine causal drug-reaction relationship
- Whether a signal warrants regulatory action
- Whether a SafetyBrief is clinically accurate
- Whether a signal should be reported to the FDA

All of these require human judgment. MedSignal surfaces evidence and prioritizes review. It does not make regulatory recommendations.

5.7 Evaluations and Testing

Overview

Our evaluation strategy has three layers. First, we validate the data pipeline against a known statistical benchmark. Second, we evaluate the agent pipeline against a golden set of ten drug-reaction pairs with documented FDA communications. Third, we assess SafetyBrief quality against a rubric that does not require domain expertise to apply. Each layer is designed to answer a specific question about whether the system is working correctly.

Pipeline Validation

PRR benchmark checkpoint The primary pipeline correctness test is the gabapentin and cardio-respiratory arrest PRR checkpoint. Before any downstream components are triggered, we confirm that this pair appears in signals_flagged with PRR clearing the threshold. Gabapentin is one of the most widely prescribed drugs in our dataset and cardio-respiratory arrest is an unambiguous MedDRA term, making this a reliable benchmark signal. This single check

validates the four-file join, caseversion deduplication, PS filter, RxNorm normalization, PRR formula, and threshold logic in one query. If this check fails, we know something in the data engineering layer is wrong and we stop before producing incorrect results.

Row count validation After Branch 1 completes, we verify that drug_reaction_pairs contains a plausible number of rows given the input data volume. A count significantly below or above the expected range indicates a join error or missing filter. Expected range for full 2023 data is approximately 4 to 6 million rows.

Signal count sanity check After Branch 2 completes, we verify that signals_flagged contains a plausible number of signals. Too few signals suggests thresholds are too strict or the quality filters are over-removing. Too many suggests a filter is not being applied. Expected range based on preliminary analysis is 1,500 to 3,000 signals.

Golden Set Evaluation

MedSignal's evaluation is grounded in a golden signal validation set, ten drug-reaction pairs for which the FDA issued verified public safety communications during 2023. These pairs were identified from 2023 FAERS data and cross-referenced against FDA Drug Safety Communications and label updates published between April 2023 and January 2024, providing a verified ground truth for evaluation. The composition of this validation set may be revised as the project progresses, drugs may be added, removed, or substituted based on data availability, signal quality, and pipeline validation outcomes.

Drug	Reaction	FDA Safety Communication
Dupixent (dupilumab)	Skin fissures / eye inflammation	FDA Label Update — January 2024
Gabapentin	Cardiorespiratory arrest	FDA Drug Safety Communication — December 2023
Pregabalin	Coma	FDA Drug Safety Communication — December 2023
Keppra (levetiracetam)	Tonic-clonic seizure	FDA Safety Communication — November 2023
Mounjaro (tirzepatide)	Hunger / injection site	FDA Drug Safety Communication — September 2023
Ozempic (semaglutide)	Increased appetite	FDA Drug Safety Communication — September 2023

Jardiance (empagliflozin)	HbA1c increased	FDA Drug Safety Communication — August 2023
Bupropion	Seizure	FDA Drug Safety Communication — May 2023
Farxiga (dapagliflozin)	GFR decreased	FDA Label Update — May 2023
Metformin	Diabetic ketoacidosis	FDA Drug Safety Communication — April 2023

The validation set serves three purposes. First, it confirms that the FAERS processing pipeline correctly identifies known pharmacovigilance signals above the PRR threshold. Second, it provides ground truth for the Evaluation Dashboard, where MedSignal's flagged signals are compared against these FDA communications as evidence of real-world detection. Third, it defines the scope of the PubMed literature corpus — load_pubmed.py fetches abstracts for each of the ten drugs, building the ChromaDB knowledge base that Agent 2 uses during signal assessment.Detection performance will be reported in the final submission once the full pipeline has been executed. The Evaluation Dashboard will display the detection lead time for each golden drug as a bar chart, and a precision-recall table will show how many of the ten golden signals MedSignal correctly flagged above the PRR threshold. FDA communication URLs are included in the project repository as verified evidence for each signal. FAERS data used for this analysis is sourced from the FDA public portal at https://fis.fda.gov/extensions/FPD-QDE-FAERS/FPD-QDE-FAERS.html

SafetyBrief Quality Evaluation

We evaluate each SafetyBrief against a four-criteria rubric that does not require pharmacovigilance expertise to apply. Two team members independently score each brief and disagreements are resolved by discussion.

Criterion	Pass condition
Signal identification	Brief correctly names the drug and reaction
Literature grounding	Every claim in brief_text is traceable to a provided abstract
Citation accuracy	All PMIDs cited in brief_text appear in the retrieved abstract set
Tier consistency	Recommended action is consistent with the assigned priority tier

A brief that passes all four criteria is counted as a quality pass. We report the pass rate across all ten golden signals as our SafetyBrief quality metric.

Hallucination detection In addition to the rubric, we check each brief for claims that cannot be traced to any provided abstract. This is a manual check for the ten golden signals. We report the number of briefs containing at least one unsupported claim as a hallucination rate.

Unit Tests

The following unit tests are written before any integration work begins.

ETL tests

- PRR formula test: given a known 2x2 contingency table, verify the formula produces the expected PRR value
- Deduplication test: given a dataset with duplicate caseids at different caseversions, verify only the highest caseversion survives
- PS filter test: given a mixed dataset of PS, SS, C, and I records, verify only PS records remain
- Normalization test: given a drug name present in the RxNorm cache, verify the canonical name is returned

Agent tests

- StatScore test: given known PRR, case count, and outcome flags, verify the expected StatScore is produced
- LitScore test: given a mock ChromaDB response with known distances, verify the expected LitScore is produced
- Priority tier test: given known StatScore and LitScore combinations, verify each of the four priority tiers is assigned correctly
- Pydantic validation test: given a malformed LLM response, verify the retry logic is triggered

Pipeline integration test A single end-to-end integration test runs the full pipeline on a small sample of 2023 Q1 data for gabapentin only, and asserts that gabapentin and cardio-respiratory arrest appears in signals_flagged with PRR above the threshold. This test is run before every demo to confirm the pipeline is intact.

5.8 Proof of Concept

Two proof-of-concept notebooks were developed during the proposal phase to validate key assumptions before building the full pipeline. Both are attached as supporting material. Numbers and results shown in these notebooks are preliminary and will be validated through the full MedSignal pipeline during the project.

POC 1 — Exploratory Data Analysis on FAERS 2023

Notebook: FAERS_POC_SingleQuarter

This notebook documents our direct examination of the 2023 Q1 FAERS ASCII files before any pipeline design decisions were made. The goal was to understand the actual data structure, field completeness, and signal characteristics before committing to an architecture.

Key findings from this analysis informed the following design decisions:

The delimiter is a dollar sign, not a pipe character as stated in most FAERS documentation. This was confirmed from raw file inspection and is applied throughout the pipeline.

The DRUG file contains 1,899,503 rows for 432,144 cases in Q1 alone. Only 22.74% of drug rows carry role_cod = PS. The PS filter is therefore essential — including concomitant drugs would inflate the PRR denominator with noise.

The prod_ai field contains the active ingredient for the majority of records, making it the primary normalization target ahead of the raw drugname field.

The OUTC file has roughly 28% of cases with no outcome recorded, confirming that the OUTC join must be a left join to avoid silently dropping cases.

The notebook also contains a preliminary PRR analysis across 2023 data that identified the ten golden drug-reaction pairs based on documented FDA communications in 2023. The lead time estimates in this analysis are preliminary and will be confirmed by the full pipeline.

POC 2 — PubMed Abstract Ingestion into ChromaDB

Notebook:
Downloading PubMed files through optimized query
Loading data into ChromaDB
Retrieving the data from ChromaDB through simulated queries

This notebook validates the PubMed fetch and ChromaDB embedding pipeline end to end for 2 drugs before scaling to all ten golden signal drugs.

The notebook demonstrates the following steps working correctly in sequence: NCBI Entrez ESearch returning up to 200 PMIDs per drug query, EFetch retrieving full XML records in batches of 20, abstract filtering removing conference abstracts and editorials, all-MiniLM-L6-v2 embedding running locally with no API cost, and ChromaDB storing vectors with drug_name and pmid metadata fields.

A retrieval validation runs a test query for dupilumab and conjunctivitis against the stored embeddings and confirms that the top returned abstracts score above 0.60 cosine similarity, validating that the threshold produces clinically relevant results for our use case. This validates that the cosine similarity threshold of 0.60 produces meaningful results for our use case.

Both notebooks are self-contained and can be run independently of the full pipeline. They require only the NCBI API key and the raw FAERS files for POC 1, and only the NCBI API key for POC 2.

Project Plan and Timeline

6.1 Milestones

Milestone 1 (M1) : Data Foundation (End of Week 1) FAERS 2023 data downloaded and schema verified. faers_prep.py publishing raw records to all four Kafka topics. RxNorm cache built and stored in PostgreSQL. Spark Branch 1 producing drug_reaction_pairs with correct row counts. Spark Branch 2 producing signals_flagged with all 10 golden drugs present. PRR benchmark checkpoint passing for gabapentin and cardio-respiratory arrest.

Milestone 2 (M2) : PubMed and ChromaDB (End of Week 1) load_pubmed.py fetching abstracts for all 10 golden signal drugs. ChromaDB loaded with approximately 1,800 to 1,930 embeddings. Retrieval validation confirming top abstracts are relevant for at least 8 of 10 drugs.

Milestone 3 (M3) : Agent Pipeline (Mid Week 2) All three LangGraph agents implemented and connected. Agent 1 generating three GPT-4o search queries per signal. Agent 2 returning top 5 abstracts above cosine threshold 0.60. Agent 3 produces Pydantic-validated SafetyBriefs for all 10 golden signals. Priority tiers assigned correctly across all signals.

Milestone 4 (M4) : Streamlit Interface (End of Week 2) All four Streamlit pages functional with real data. Signal feed and evaluation dashboard owned by Samiksha. Signal detail page owned by Prachi. HITL queue owned by Siddharth. All pages connecting correctly to PostgreSQL and ChromaDB.

Milestone 5 (M5) : Evaluation and Validation (Mid Week 3) Detection lead time computed for all 10 golden signals. SafetyBrief quality rubric applied independently by two team members. Hallucination check completed. Unit tests written and passed for all components. Each member writes tests for the components they built.

Milestone 6 (M6) : Final Polish and Demo Prep (End of Week 3) Full pipeline run on complete 2023 data with metrics captured. Prometheus dashboard live. Proposal complete. Demo rehearsed with all three members.

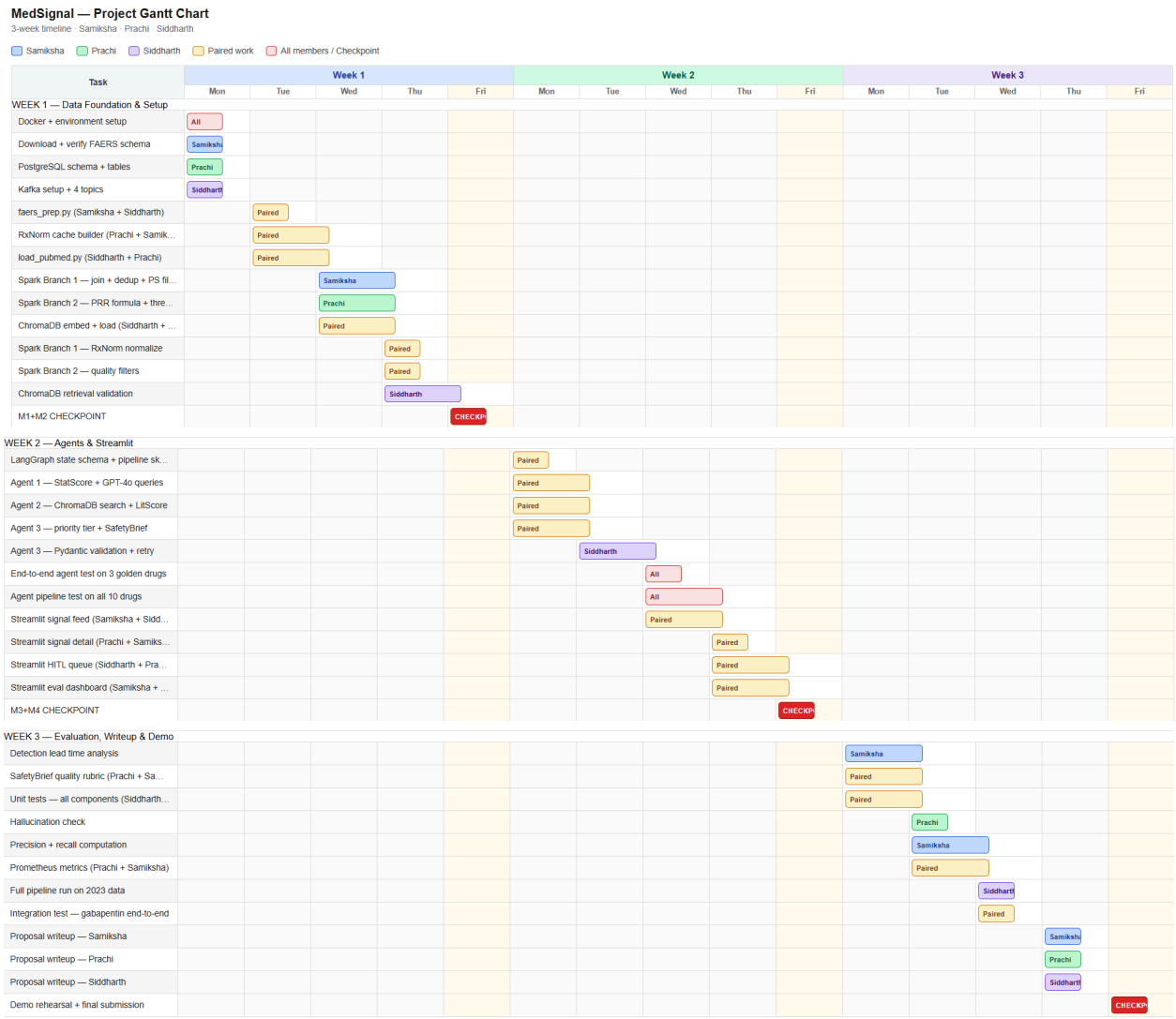
6.2 Timeline

Each member is assigned a primary ownership area and is accountable for its delivery, irrespective of who co-builds it. Tasks are deliberately paired so that every team member gets hands-on experience with Kafka, Spark, PostgreSQL, LangGraph, ChromaDB, and Streamlit across the three weeks. No one is siloed into a single tool for the entire project.

Samiksha — Data Engineering Lead is responsible for: data ingestion pipeline, Spark Branch 1, Agent 1, Streamlit signal feed and evaluation dashboard, detection lead time evaluation.

Prachi — Data Processing and LLM Lead is responsible for: RxNorm cache and PostgreSQL schema, Spark Branch 2, Agent 2, Streamlit signal detail page, SafetyBrief quality evaluation.

Siddharth — Infrastructure and Quality Lead is responsible for: Kafka setup, ChromaDB and PubMed pipeline, Agent 3, Streamlit HITL queue, unit tests for all components.



Team Roles and Responsibilities

MedSignal is built by a three-person team over three weeks. Each member holds a lead title reflecting their primary area of accountability. Tasks are paired across members so that everyone gains hands-on experience with every major tool in the stack: Kafka, Spark, PostgreSQL, LangGraph, ChromaDB, and Streamlit. No one is confined to a single component for the duration of the project.

Samiksha — Data Engineering Lead

Primary responsibilities:

- FAERS 2023 data download, schema verification, and field completeness analysis
- faers_prep.py - thin Kafka producer reading raw ASCII files and publishing to four topics
- Spark Branch 1 - four-file join, caseversion deduplication, PS filter, RxNorm broadcast normalization
- Agent 1 - StatScore computation and GPT-4o search query generation
- Streamlit signal feed page and evaluation dashboard
- Detection lead time analysis across all 10 golden signal drugs

Co-builds with:

- Siddharth on faers_prep.py and ChromaDB loading
- Prachi on RxNorm normalization, Agent 1, SafetyBrief quality rubric, and Prometheus metrics

Prachi — Data Processing and LLM Lead

Primary responsibilities:

- PostgreSQL schema design - all four tables with indexes
- RxNorm cache builder - NIH API calls, canonical name resolution, PostgreSQL storage
- Spark Branch 2 - PRR formula, threshold filters, quality filters (junk term, spike, late-surge)
- Agent 2 - ChromaDB cosine search, LitScore computation
- Streamlit signal detail page
- SafetyBrief quality rubric evaluation and hallucination check
- Precision and recall computation across full signals_flagged

Co-builds with:

- Samiksha on RxNorm normalization, Agent 1, SafetyBrief rubric, and Prometheus metrics
- Siddharth on load_pubmed.py, Spark quality filters, LangGraph pipeline skeleton, HITL queue, and unit tests

Siddharth — Infrastructure and Quality Lead

Primary responsibilities:

- Kafka setup - four topics, producer connectivity, Docker configuration
- load_pubmed.py - NCBI ESearch and EFetch, abstract filtering, metadata tagging
- ChromaDB - all-MiniLM-L6-v2 embedding, vector storage, cosine threshold tuning
- Agent 3 - priority tier assignment, GPT-4o SafetyBrief prompt, Pydantic validation and retry logic
- Streamlit HITL queue - priority-sorted display, approve/reject/escalate, hitl_decisions write
- Unit tests for all pipeline components
- Prometheus metrics and pipeline observability

- Full pipeline run on complete 2023 data

Co-builds with:

- Samiksha on faers_prep.py, ChromaDB loading, Agent 3, signal feed, and integration test
- Prachi on load_pubmed.py, Spark quality filters, LangGraph pipeline skeleton, and HITL queue

Shared Responsibilities

The following activities involve all three members equally:

- Docker environment setup on Day 1
- End-to-end agent pipeline testing on 3 golden drugs (Week 2 Wednesday)
- Full agent pipeline test on all 10 golden drugs (Week 2 Thursday)
- Demo rehearsal on Week 3 Friday — every member must be able to answer questions on any component of the system

Risks and Mitigation

8.1 Potential Risks

Risk 1 — NCBI API Rate Limiting and Fetch Failures

The PubMed download pipeline (load_pubmed.py) makes repeated calls to the NCBI E-utilities API to retrieve up to 200 abstracts per drug. Without an API key, the rate limit is three requests per second. Temporary 500-series server errors from NCBI were observed during testing for the semaglutide download batch, resulting in partial abstract loss for that drug. The pipeline is also susceptible to network interruptions that could leave ChromaDB in a partially loaded state.

Risk 2 — FAERS Data Quality and Inconsistency

FAERS is a spontaneous reporting system and is not exhaustive. Drug names are reported verbatim by submitters, producing substantial fragmentation across brand names, generic names, dose variants, and typos. Additionally, 38% of DEMO records are missing the event date field, and 28% of cases have no recorded outcome code, introducing systematic gaps that affect both PRR stability and severity scoring.

Risk 3 — LLM Hallucination and Citation Fabrication

Agent 3 uses GPT-4o to synthesize a SafetyBrief from retrieved abstracts and statistical evidence. Large language models are known to fabricate plausible-sounding but non-existent citations when generating text about scientific topics. A SafetyBrief containing a fabricated PMID would mislead the human reviewer and undermine the clinical credibility of the system. This risk is elevated when ChromaDB retrieval returns few high-similarity abstracts, leaving Agent 3 with insufficient grounding context.

Risk 4 — GPT-4o API Cost

GPT-4o is called twice per signal: once in Agent 1 to generate three search query strings, and once in Agent 3 to synthesize the SafetyBrief. The Agent 3 prompt is token-intensive because it includes the full text of five retrieved abstracts alongside the signal statistics. Although the agent pipeline is scoped to the ten golden signal drugs only, the retry logic that fires a second Agent 3 call if Pydantic validation fails on the first response could increase costs during development if validation failures are frequent.

Risk 5 — Spark Memory and Processing Constraints

The four-quarter FAERS dataset produces approximately five million rows in the drug_reaction_pairs table after Branch 1 completes. The PRR aggregation in Branch 2 requires groupBy operations across all drug-reaction combinations, which involves a full shuffle across Spark partitions. Running Spark in local mode on a laptop with 16 GB of RAM creates a risk of memory exhaustion during the shuffle step, particularly when the broadcast RxNorm cache and the join dataframes are simultaneously in memory.

8.2 Mitigation Strategies

Rate Limiting, Retries, and Incremental Loading (Risk 1)

Each team member uses a personal NCBI API key, raising the per-key rate limit from three to ten requests per second. The download script sleeps 0.12 seconds between API calls and implements a single automatic retry with a five-second backoff on any 500-series server error. Batches are fetched in groups of twenty PMIDs per EFetch call, consistent with NCBI recommended batch sizes. ChromaDB uses a persistent client with incremental collection.add() calls, so a partial run can be resumed without reprocessing already-loaded records. PMID deduplication within each drug ensures that retried batches do not create duplicate entries.

RxNorm Normalization and Data Quality Guardrails (Risk 2)

Drug name fragmentation is addressed through a three-level normalization hierarchy in Spark Branch 1: prod_ai active ingredient field as the primary source, RxNorm canonical name from the pre-built PostgreSQL cache as fallback, and raw drugname as a last resort. The RxNorm cache is built once before the Spark job runs using the NIH RxNorm REST API, eliminating live API calls during distributed processing. The cache is broadcast to all Spark executors as a read-only variable to avoid repeated PostgreSQL lookups. The OUTC join is implemented as a left join with null outcome flags filled to zero, ensuring cases with no recorded outcome are retained rather than silently dropped. Three post-PRR quality filters remove junk MedDRA terms, single-quarter reporting spikes, and late-surge signals driven by media attention rather than organic accumulation.

Citation Validation and RAG Grounding (Risk 3)

Hallucination is mitigated at two levels. First, the Agent 3 prompt injects the verbatim abstract text of the five retrieved papers directly as context and instructs the model to use only that text

for clinical claims, rather than drawing on parametric training knowledge. Second, a citation validator post-processes every SafetyBrief output and checks each cited PMID against the abstracts retrieved by Agent 2. Any PMID cited in the output that was not in the five retrieved abstracts is flagged as a hallucination and the SafetyBrief is rejected, triggering a retry. The SafetyBrief schema is enforced by Pydantic v2 before any output is written to PostgreSQL, catching structural errors before they reach the reviewer.

Scoped Agent Execution and Embedding Caching (Risk 4)

GPT-4o API cost is controlled by scoping the agent pipeline to signals involving the ten golden signal drugs only. All PubMed embeddings are generated once during the data preparation phase using the locally-running all-MiniLM-L6-v2 model and cached persistently in ChromaDB. Agent 2 incurs no embedding API cost at query time. A hard spend limit of $10 is set on the OpenAI account before any pipeline run begins, providing a buffer well above the estimated total cost of approximately $0.25 using GPT-4o mini during development and GPT-4o for the final evaluation run.

Distributed Compute and Scoped Data Volume (Risk 5)

The Spark pipeline is scoped to four quarters of 2023 FAERS data, keeping the in-memory footprint within the bounds of a 16 GB development machine. The RxNorm cache is broadcast as a read-only variable so all executors share a single copy. The four-file join is structured to filter the DRUG dataframe to PS records only before the join, reducing the join input size by approximately 78%. The Spark architecture is designed to scale horizontally to larger data volumes by adding workers, and the same pipeline code runs unchanged on a cloud Spark cluster if local resources prove insufficient.

Expected Outcomes and Metrics

9.1 Key Performance Indicators

KPI 1 — Signal Detection Accuracy

Target: at least 8 of the 10 golden drug-reaction signals correctly flagged by the PRR pipeline above threshold (PRR >= 2.0, A >= 50). This metric is measured from the signals_flagged table against the known golden signal list.

KPI 2 — Detection Lead Time

Target: positive detection lead time for each correctly flagged golden signal, measured as the difference in days between the date MedSignal first flags the signal in 2023 FAERS data and the documented FDA safety communication date. Preliminary POC analysis identified lead times ranging from 13 days (metformin) to 291 days (dupilumab), with a median of approximately 175 days across the ten signals. The full Spark pipeline is expected to confirm these estimates with higher PRR stability across four quarters.

KPI 3 — Spark Processing Runtime

Target: Spark batch job completes Branch 1 and Branch 2 for all four 2023 quarters in under three hours on the development machine in local mode.

KPI 4 — ChromaDB Retrieval Quality

Target: at least three of the five abstracts returned by Agent 2 score above the 0.60 cosine similarity threshold for each of the ten golden drug-reaction pairs. This threshold was calibrated empirically against the PubMed corpus using all-MiniLM-L6-v2. Preliminary retrieval testing confirmed that natural language queries generated by Agent 1 produce similarity scores in the range of 0.61 to 0.76 for the dupilumab and semaglutide test cases. These results are preliminary and will be confirmed by the full pipeline.

KPI 5 — Citation Validation Pass Rate

Target: zero fabricated PMIDs in any SafetyBrief written to PostgreSQL. The citation validator rejects any SafetyBrief containing a PMID not present in the five retrieved abstracts passed to Agent 3. A 100% citation validation pass rate across all ten golden signals is the minimum acceptable standard.

9.2 Expected Benefits

MedSignal addresses a specific and documented gap in the pharmacovigilance signal triage workflow. Every existing system including the WHO Uppsala Monitoring Centre's VigiBase, the FDA's internal Empirical Bayes Geometric Mean tool, and commercial platforms including Oracle Argus and Veeva Vault Safety produces a statistical signal score and stops there. The downstream investigation step, which involves retrieving relevant literature, assessing biological plausibility, and producing a prioritized narrative for human review, is performed entirely by hand. For agencies processing thousands of signals per quarter, this manual step is the primary bottleneck determining how quickly a signal can be acted upon.

MedSignal closes this gap by automating the literature retrieval and narrative synthesis steps while preserving human judgment at the final decision point. A pharmacovigilance analyst reviewing signals in the HITL queue sees a prioritized SafetyBrief that includes the statistical evidence, the supporting literature with cited PMIDs, and a mechanism hypothesis drawn from published abstracts. The analyst makes an approve, reject, or escalate decision rather than performing the underlying research from scratch. This shifts the analyst's role from information gatherer to clinical decision maker, which is where pharmacovigilance expertise is most valuable.

From a technical standpoint, MedSignal demonstrates that a distributed big data pipeline and a citation-grounded LLM agent system can be combined into a single coherent architecture deployable on commodity hardware. The Spark layer handles the scale requirements of the FAERS dataset without requiring cloud infrastructure. The RAG architecture ensures that LLM outputs are grounded in verifiable published evidence rather than uncontrolled parametric generation. The HITL gate ensures that no signal is acted upon without a qualified human decision, which is both a regulatory requirement and a safety property of the system.

The detection lead time analysis provides an empirically grounded evaluation of the system's value. Preliminary POC analysis identified a median lead time of approximately 175 days across the ten golden signals, subject to confirmation by the full validated pipeline. This figure should be interpreted carefully: the golden signal set was constructed by selecting drug-reaction pairs with documented FDA communications in 2023, meaning these signals were chosen in part because they were already detectable in FAERS data. Detection lead time is therefore a necessary but not sufficient measure of system value it must be read alongside the precision metric from the evaluation dashboard. Earlier detection is only clinically meaningful if the false positive rate across the broader signals_flagged table remains acceptably low. Subject to that caveat, a confirmed positive lead time across the majority of golden signals would demonstrate that automated PRR-based detection on routinely available FAERS data can meaningfully accelerate the pharmacovigilance review cycle. For drug safety signals with the severity profile of those in the golden set including signals associated with suicidality, respiratory arrest, and cardiac events — even a fraction of that lead time advantage represents a potentially significant clinical and public health benefit.

Token and Cost Report

Overview

MedSignal makes GPT-4o API calls at two points in the agent pipeline. All other components StatScore computation, ChromaDB retrieval, LitScore calculation, Pydantic validation run locally with no API cost. Token usage is predictable and bounded because both GPT-4o calls have fixed input structures and capped output lengths.

GPT-4o Usage Points

Agent 1 — Search Query Generation

Each Agent 1 call receives a short structured prompt containing the drug name, reaction term, PRR value, and case count. It returns exactly three search queries as a JSON array.

Estimated tokens per call:
System prompt: ~80 tokens
User message: ~60 tokens
Response: ~80 tokens
Total per call: ~220 tokens

Calls per pipeline run: 10 (one per golden signal drug)
Total tokens per run: ~2,200 tokens

Agent 3 — SafetyBrief Generation

Each Agent 3 call receives the signal statistics, StatScore, LitScore, priority tier, and up to five retrieved abstracts with PMIDs. It returns a structured JSON SafetyBrief.

Estimated tokens per call:
System prompt: ~120 tokens
Signal statistics: ~80 tokens
5 abstracts + PMIDs: ~1,500 tokens
Response: ~400 tokens
Total per call: ~2,100 tokens

Calls per pipeline run: 10 (one per golden signal drug)
Total tokens per run: ~21,000 tokens

Total Token Estimate

Activity	Calls	Tokens per call	Total tokens
Agent 1 — search queries	10	~220	~2,200
Agent 3 — SafetyBriefs	10	~2,100	~21,000
Retry calls (estimated 10%)	~2	~2,100	~4,200
Development and testing runs	~20 runs	~23,200	~464,000
Final evaluation run	**1 run**	**~23,200**	**~23,200**
Total estimated			**~490,000 tokens**

Development runs dominate the total because the pipeline is run repeatedly during testing and debugging. The final evaluation run is a small fraction of total usage.

Cost Estimate

GPT-4o pricing as of the proposal date is $5.00 per million input tokens and $15.00 per million output tokens. Using a blended estimate of approximately 80% input and 20% output:

Input tokens: ~392,000 × $5.00 / 1,000,000 = ~$1.96
Output tokens: ~98,000 × $15.00 / 1,000,000 = ~$1.47
Total estimated cost: ~$3.43

We set a hard spend limit of $10 on the OpenAI account before any pipeline run begins. This provides a 3x buffer above the estimated cost and ensures no unexpected charges during development.

Cost Reduction Strategy

GPT-4o mini during development All development and debugging runs use GPT-4o mini instead of GPT-4o. GPT-4o mini costs approximately $0.15 per million input tokens and $0.60 per million output tokens — roughly 30x cheaper. Only the final evaluation run and demo use GPT-4o.

Development cost with GPT-4o mini:
~466,800 tokens × blended $0.20/1M = ~$0.09

Final run with GPT-4o:
~23,200 tokens × blended $7.00/1M = ~$0.16

Total realistic cost: ~$0.25

Prompt caching Agent 1 system prompt and Agent 3 system prompt are identical across all 10 signal calls within a single pipeline run. OpenAI's prompt caching reduces the cost of repeated identical prefixes by approximately 50%. This applies to both the system prompt and the static portions of the user message.

Abstract truncation Each abstract is truncated to 300 words before being included in the Agent 3 prompt. Full abstracts average 250 to 300 words so truncation rarely activates, but it prevents outlier long abstracts from inflating token counts.

Batching All 10 golden signal calls are run sequentially within a single pipeline execution rather than triggered individually. This keeps API usage predictable and makes cost tracking straightforward.

Monitoring

Token usage per call is logged to PostgreSQL in the safety_briefs table using the usage field returned by the OpenAI API response. After each pipeline run, a summary query reports total tokens consumed, estimated cost, and any calls that triggered the retry logic. This gives the team full visibility into actual versus estimated usage.

Conclusion

Drug safety surveillance is a problem that sits at the intersection of public health and data engineering. The FDA's FAERS database receives millions of adverse event reports every year, but the signals buried in that data can take months to surface through manual review processes. MedSignal is our attempt to close that gap not by replacing human judgment, but by giving pharmacovigilance analysts better tools to find and act on signals faster.

What we set out to build is an end-to-end pipeline that takes raw quarterly FAERS data, processes it through a Kafka and Spark batch architecture, detects statistically significant drug-reaction signals using Proportional Reporting Ratio computation, and passes those signals

through a three-agent LLM pipeline that retrieves supporting literature from PubMed and generates structured safety briefs for human review.

The system is evaluated against ten golden drug-reaction pairs with documented FDA safety communications in 2023. Our preliminary analysis identified these signals in FAERS data weeks to months before the FDA formally communicated them, with detection lead times. The full pipeline validation during the project will confirm or refine these results.

Beyond the technical deliverable, this project gave us the opportunity to work deeply with real regulatory data examining actual FAERS files, understanding their schema and noise characteristics, and designing a pipeline that handles the messiness of real-world adverse event reporting. That data engineering foundation is what makes the LLM layer meaningful. A system that surfaces unreliable signals for LLM synthesis is worse than no system at all. We built the statistical layer first, validated it against known benchmarks, and only then connected it to the agent pipeline.

The architecture is designed with production extensibility in mind. The same Kafka and Spark pipeline that processes four quarters of historical FAERS data can be connected to live hospital EHR feeds or manufacturer expedited reports without changing any downstream components. The ChromaDB index can be updated incrementally as new literature is published. The HITL queue can scale to larger reviewer teams. MedSignal as built is a research prototype, but the engineering decisions reflect what a production pharmacovigilance system would require.

References

Datasets

U.S. Food and Drug Administration. (2023). *FDA Adverse Event Reporting System (FAERS)*. https://fis.fda.gov/extensions/FPD-QDE-FAERS/FPD-QDE-FAERS.html

National Library of Medicine. (2023). *PubMed / NCBI Entrez API*. https://www.ncbi.nlm.nih.gov/home/develop/api/

National Library of Medicine. (2023). *RxNorm API*. https://rxnav.nlm.nih.gov/

FDA Safety Communications

U.S. Food and Drug Administration. (2023, April 13). *FDA drug safety communication: Metformin diabetic ketoacidosis*. https://www.fda.gov/drugs/drug-safety-and-availability

U.S. Food and Drug Administration. (2023, May 9). *FDA drug safety communication: Farxiga (dapagliflozin) GFR decreased and cardiovascular death*. https://www.fda.gov/drugs/drug-safety-and-availability

U.S. Food and Drug Administration. (2023, May 11). *FDA drug safety communication: Bupropion seizure*. https://www.fda.gov/drugs/drug-safety-and-availability

U.S. Food and Drug Administration. (2023, August 1). *FDA drug safety communication: Jardiance (empagliflozin) HbA1c and DKA signal*. https://www.fda.gov/drugs/drug-safety-and-availability

U.S. Food and Drug Administration. (2023, September 22). *FDA drug safety communication: Mounjaro (tirzepatide) and Ozempic (semaglutide) — injection reactions and appetite/thyroid signal*. https://www.fda.gov/drugs/drug-safety-and-availability

U.S. Food and Drug Administration. (2023, November 28). *FDA drug safety communication: Levetiracetam — tonic-clonic seizure*. https://www.fda.gov/drugs/drug-safety-and-availability

U.S. Food and Drug Administration. (2023, December 19). *FDA drug safety communication: Gabapentin and pregabalin — risk of serious breathing problems*. https://www.fda.gov/drugs/drug-safety-and-availability

U.S. Food and Drug Administration. (2024, January 16). *FDA drug safety communication: Dupixent (dupilumab) — skin fissures and eye irritation*. https://www.fda.gov/drugs/drug-safety-and-availability

Pharmacovigilance Methodology

Evans, S. J. W., Waller, P. C., & Davis, S. (2001). Use of proportional reporting ratios (PRRs) for signal generation from spontaneous adverse drug reaction reports. *Pharmacoepidemiology and Drug Safety*, *10*(6), 483–486. https://doi.org/10.1002/pds.677

Bate, A., & Evans, S. J. W. (2009). Quantitative signal detection using spontaneous ADR reporting. *Pharmacoepidemiology and Drug Safety*, *18*(6), 427–436. https://doi.org/10.1002/pds.1742

Technologies

Apache Software Foundation. (2023). *Apache Kafka* (Version 3.x) [Software]. https://kafka.apache.org

Apache Software Foundation. (2023). *Apache Spark* (Version 3.x) [Software]. https://spark.apache.org

LangChain. (2024). *LangGraph* [Software]. https://github.com/langchain-ai/langgraph

Chroma. (2024). *ChromaDB* [Software]. https://www.trychroma.com

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

OpenAI. (2024). *GPT-4o* [Large language model]. https://platform.openai.com

Pydantic. (2024). *Pydantic V2* [Software]. https://docs.pydantic.dev

Streamlit Inc. (2024). *Streamlit* [Software]. https://streamlit.io

Prometheus Authors. (2024). *Prometheus* [Software]. https://prometheus.io